

Pontificia Universidade Catolica de Goias

Escola Politecnica e de Artes
Curso de Ciencia da Computacao

Atividade Final

Paradigmas de Linguagens de Programacao

Sistema de Analise de Desempenho Academico

Alunos:

Amanda Barbosa Ferreira Agapito
Guilherme Jose da Silva
Matheus Silva Pains

Professor: Roussian de Ramos Alves Gaioso

Goiania, Goias
12 de Junho de 2026

1. Descricao do Problema

O sistema recebe dados de alunos contendo nome, matricula, tres notas e frequencia. A media e calculada como: $media = (nota1 + nota2 + nota3) / 3$. O aluno e aprovado se $media \geq 6.0$ E frequencia $\geq 75\%$. Caso contrario, e reprovado.

Dados de Entrada (alunos.csv)

```
Ana;2024001;8.0;7.5;9.0;85
Bruno;2024002;5.0;6.0;4.5;80
Carlos;2024003;7.0;6.5;8.0;60
Daniela;2024004;9.0;9.5;10.0;95
Amanda;2024005;10;9;10;100
Guilherme;2024006;9;10;10;100
Matheus;2024007;10;10;9;100
```

Resultado Esperado

Ana: media 8.17, freq 85 -> Aprovado | Bruno: media 5.17, freq 80 -> Reprovado

Carlos: media 7.17, freq 60 -> Reprovado | Daniela: media 9.50, freq 95 -> Aprovado

Amanda: media 9.67, freq 100 -> Aprovado | Guilherme: media 9.67, freq 100 -> Aprovado

Matheus: media 9.67, freq 100 -> Aprovado

2. Linguagens Escolhidas

- C (imperativo) -- linguagem procedural por excelencia, com controle explicito de memoria, structs, ponteiros e funcoes. Ideal para o paradigma imperativo puro.

- Python (orientado a objetos) -- classes com encapsulamento (name mangling), metodos de negocio e organizacao em objetos.

- Python (funcional) -- map, filter, reduce, lambdas e dicionarios imutaveis para um estilo predominantemente funcional.

- Prolog (logico) -- linguagem declarativa baseada em fatos e regras.

C foi escolhida para a versao imperativa por ser a linguagem sugerida pelo enunciado e a mais representativa do paradigma procedural. Python foi utilizada nas versoes OO e funcional por sua versatilidade. Prolog foi mantido por ser a referencia natural do paradigma logico.

3. Parte 1 -- Implementacao Imperativa (C)

Codigo-fonte: parte1_imperativo.c

```

1 | /*
2 |  * Parte 1 - Implementacao Imperativa (C)
3 |  * Sistema de Analise de Desempenho Academico
4 |  */
5 |
6 | #include <stdio.h>
7 | #include <stdlib.h>
8 | #include <string.h>
9 |
10 | int main() {
11 |     FILE *arquivo;
12 |     char linha[200];
13 |     char nome[50];
14 |     char matricula[20];
15 |     float nota1, nota2, nota3;
16 |     int frequencia;
17 |     float media;
18 |     int total = 0;
19 |     int aprovados = 0;
20 |     int reprovados = 0;
21 |
22 |     arquivo = fopen("alunos.csv", "r");
23 |     if (arquivo == NULL) {
24 |         printf("Erro ao abrir o arquivo!\n");
25 |         return 1;
26 |     }
27 |
28 |     printf("=====\n");
29 |     printf("RELATORIO DE DESEMPENHO ACADEMICO - VERSAO IMPERATIVA (C)\n");
30 |     printf("=====\n");
31 |     printf("%-12s %-12s %-6s %-6s %-6s %-8s %-6s %s\n",
32 |           "Nome", "Matricula", "N1", "N2", "N3", "Media", "Freq", "Situacao");
33 |     printf("-----\n");
34 |
35 |     while (fgets(linha, 200, arquivo) != NULL) {
36 |         linha[strcspn(linha, "\r\n")] = '\0';
37 |
38 |         if (strlen(linha) == 0)
39 |             continue;
40 |
41 |         /* separa os campos */
42 |         char *campo = strtok(linha, ";");
43 |         strcpy(nome, campo);
44 |
45 |         campo = strtok(NULL, ";");
46 |         strcpy(matricula, campo);
47 |
48 |         campo = strtok(NULL, ";");
49 |         nota1 = atof(campo);
50 |
51 |         campo = strtok(NULL, ";");
52 |         nota2 = atof(campo);
53 |
54 |         campo = strtok(NULL, ";");
55 |         nota3 = atof(campo);
56 |
57 |         campo = strtok(NULL, ";");
58 |         frequencia = atoi(campo);
59 |
60 |         /* calcula a media */
61 |         media = (nota1 + nota2 + nota3) / 3.0;
62 |
63 |         /* verifica aprovacao */
64 |         if (media >= 6.0 && frequencia >= 75) {

```

```

65 |         printf("%-12s %-12s %-6.1f %-6.1f %-6.1f %-8.2f %-6d Aprovado\n",
66 |             nome, matricula, nota1, nota2, nota3, media, frequencia);
67 |         aprovados++;
68 |     } else {
69 |         printf("%-12s %-12s %-6.1f %-6.1f %-6.1f %-8.2f %-6d Reprovado\n",
70 |             nome, matricula, nota1, nota2, nota3, media, frequencia);
71 |         reprovados++;
72 |     }
73 |
74 |     total++;
75 | }
76 |
77 | fclose(arquivo);
78 |
79 | printf("-----\n");
80 | printf("Total de alunos: %d\n", total);
81 | printf("Aprovados: %d\n", aprovados);
82 | printf("Reprovados: %d\n", reprovados);
83 | printf("=====\n");
84 |
85 | return 0;
86 | }

```

Saida da Execucao

```

=====
RELATORIO DE DESEMPENHO ACADEMICO - VERSAO IMPERATIVA (C)
=====
Nome          Matricula    N1     N2     N3     Media    Freq    Situacao
-----
Ana           2024001      8.0    7.5    9.0    8.17     85     Aprovado
Bruno         2024002      5.0    6.0    4.5    5.17     80     Reprovado
Carlos        2024003      7.0    6.5    8.0    7.17     60     Reprovado
Daniela       2024004      9.0    9.5    10.0   9.50     95     Aprovado
Amanda        2024005     10.0    9.0    10.0   9.67    100     Aprovado
Guilherme     2024006      9.0    10.0   10.0   9.67    100     Aprovado
Matheus       2024007     10.0    10.0   9.0    9.67    100     Aprovado
-----
Total de alunos: 7
Aprovados: 5
Reprovados: 2
=====

```

4. Parte 2 -- Implementacao Orientada a Objetos (Python)

Codigo-fonte: parte2_oo.py

```
1 | # Parte 2 - Implementacao Orientada a Objetos (Python)
2 | # Sistema de Analise de Desempenho Academico
3 |
4 | import os
5 |
6 |
7 | class Aluno:
8 |     def __init__(self, nome, matricula, nota1, nota2, nota3, frequencia):
9 |         self.nome = nome
10 |        self.matricula = matricula
11 |        self.nota1 = nota1
12 |        self.nota2 = nota2
13 |        self.nota3 = nota3
14 |        self.frequencia = frequencia
15 |
16 |    def calcular_media(self):
17 |        return (self.nota1 + self.nota2 + self.nota3) / 3
18 |
19 |    def verificar_aprovacao(self):
20 |        media = self.calcular_media()
21 |        if media >= 6.0 and self.frequencia >= 75:
22 |            return "Aprovado"
23 |        else:
24 |            return "Reprovado"
25 |
26 |
27 | class Turma:
28 |     def __init__(self):
29 |         self.alunos = []
30 |
31 |    def carregar_arquivo(self, caminho):
32 |        arquivo = open(caminho, "r", encoding="utf-8")
33 |        for linha in arquivo:
34 |            linha = linha.strip()
35 |            if linha == "":
36 |                continue
37 |            partes = linha.split(";")
38 |            aluno = Aluno(
39 |                partes[0],
40 |                partes[1],
41 |                float(partes[2]),
42 |                float(partes[3]),
43 |                float(partes[4]),
44 |                int(partes[5])
45 |            )
46 |            self.alunos.append(aluno)
47 |        arquivo.close()
48 |
49 |    def gerar_relatorio(self):
50 |        print("=" * 70)
51 |        print("RELATORIO DE DESEMPENHO ACADEMICO - VERSAO 00 (Python)")
52 |        print("=" * 70)
53 |        print(f"{'Nome':<12} {'Matricula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'Media':<8} {'Freq':<6} ...")
54 |        print("-" * 70)
55 |
56 |        aprovados = 0
57 |        reprovados = 0
58 |
59 |        for aluno in self.alunos:
60 |            media = aluno.calcular_media()
61 |            situacao = aluno.verificar_aprovacao()
62 |            print(f"{aluno.nome:<12} {aluno.matricula:<12} {aluno.nota1:<6.1f} {aluno.nota2:<6.1f} {al...")
63 |
64 |            if situacao == "Aprovado":
```

```

65 |         aprovados += 1
66 |     else:
67 |         reprovados += 1
68 |
69 |     print("-" * 70)
70 |     print(f"Total de alunos: {len(self.alunos)}")
71 |     print(f"Aprovados: {aprovados}")
72 |     print(f"Reprovados: {reprovados}")
73 |     print("=" * 70)
74 |
75 |
76 | # Programa principal
77 | caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "alunos.csv")
78 | turma = Turma()
79 | turma.carregar_arquivo(caminho)
80 | turma.gerar_relatorio()

```

Saida da Execucao

```

=====
RELATORIO DE DESEMPENHO ACADEMICO - VERSAO 00 (Python)
=====
Nome           Matricula    N1      N2      N3      Media    Freq    Situacao
-----
Ana            2024001      8.0     7.5     9.0     8.17     85     Aprovado
Bruno          2024002      5.0     6.0     4.5     5.17     80     Reprovado
Carlos         2024003      7.0     6.5     8.0     7.17     60     Reprovado
Daniela        2024004      9.0     9.5     10.0    9.50     95     Aprovado
Amanda         2024005      10.0    9.0     10.0    9.67     100    Aprovado
Guilherme      2024006      9.0     10.0    10.0    9.67     100    Aprovado
Matheus        2024007      10.0    10.0    9.0     9.67     100    Aprovado
-----
Total de alunos: 7
Aprovados: 5
Reprovados: 2
=====

```

5. Parte 3 -- Implementacao Funcional (Python)

Codigo-fonte: parte3_funcional.py

```
1 | # Parte 3 - Implementacao Funcional (Python)
2 | # Sistema de Analise de Desempenho Academico
3 |
4 | import os
5 | from functools import reduce
6 |
7 |
8 | # Funcoes puras
9 |
10 | def parse_linha(linha):
11 |     partes = linha.strip().split(";")
12 |     return {
13 |         "nome": partes[0],
14 |         "matricula": partes[1],
15 |         "nota1": float(partes[2]),
16 |         "nota2": float(partes[3]),
17 |         "nota3": float(partes[4]),
18 |         "frequencia": int(partes[5])
19 |     }
20 |
21 |
22 | def calcular_media(aluno):
23 |     return (aluno["nota1"] + aluno["nota2"] + aluno["nota3"]) / 3
24 |
25 |
26 | def verificar_aprovacao(aluno):
27 |     return calcular_media(aluno) >= 6.0 and aluno["frequencia"] >= 75
28 |
29 |
30 | def gerar_resultado(aluno):
31 |     media = calcular_media(aluno)
32 |     situacao = "Aprovado" if verificar_aprovacao(aluno) else "Reprovado"
33 |     return {
34 |         "nome": aluno["nome"],
35 |         "matricula": aluno["matricula"],
36 |         "nota1": aluno["nota1"],
37 |         "nota2": aluno["nota2"],
38 |         "nota3": aluno["nota3"],
39 |         "frequencia": aluno["frequencia"],
40 |         "media": media,
41 |         "situacao": situacao
42 |     }
43 |
44 |
45 | def contar_situacao(acc, r):
46 |     if r["situacao"] == "Aprovado":
47 |         return (acc[0] + 1, acc[1])
48 |     else:
49 |         return (acc[0], acc[1] + 1)
50 |
51 |
52 | # Entrada
53 | caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "alunos.csv")
54 | arquivo = open(caminho, "r", encoding="utf-8")
55 | linhas = arquivo.readlines()
56 | arquivo.close()
57 |
58 | alunos = list(map(parse_linha, filter(lambda l: l.strip() != "", linhas)))
59 |
60 | # Processamento com map
61 | resultados = list(map(gerar_resultado, alunos))
62 |
63 | # Filter para separar aprovados
64 | aprovados = list(filter(lambda r: r["situacao"] == "Aprovado", resultados))
```

```

65 |
66 | # Saida
67 | print("=" * 70)
68 | print("RELATORIO DE DESEMPENHO ACADEMICO - VERSAO FUNCIONAL (Python)")
69 | print("=" * 70)
70 | print(f"{'Nome':<12} {'Matricula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'Media':<8} {'Freq':<6} {'Situac...
71 | print("-" * 70)
72 |
73 | for r in resultados:
74 |     print(f"{r['nome']:<12} {r['matricula']:<12} {r['nota1']:<6.1f} {r['nota2']:<6.1f} {r['nota3']:<6....
75 |
76 | # Reduce para contar
77 | total_aprov, total_reprov = reduce(contar_situacao, resultados, (0, 0))
78 |
79 | print("-" * 70)
80 | print(f"Total de alunos: {len(resultados)}")
81 | print(f"Aprovados: {total_aprov}")
82 | print(f"Reprovados: {total_reprov}")
83 | print("=" * 70)
84 |
85 | print("\n--- Aprovados (via filter) ---")
86 | for a in aprovados:
87 |     print(f" {a['nome']} - Media: {a['media']:.2f}")

```

Saida da Execucacao

```

=====
RELATORIO DE DESEMPENHO ACADEMICO - VERSAO FUNCIONAL (Python)
=====
Nome           Matricula    N1     N2     N3     Media    Freq    Situacao
-----
Ana            2024001      8.0    7.5    9.0    8.17     85      Aprovado
Bruno          2024002      5.0    6.0    4.5    5.17     80      Reprovado
Carlos         2024003      7.0    6.5    8.0    7.17     60      Reprovado
Daniela        2024004      9.0    9.5    10.0   9.50     95      Aprovado
Amanda         2024005      10.0   9.0    10.0   9.67     100     Aprovado
Guilherme      2024006      9.0    10.0   10.0   9.67     100     Aprovado
Matheus        2024007      10.0   10.0   9.0    9.67     100     Aprovado
-----
Total de alunos: 7
Aprovados: 5
Reprovados: 2
=====

--- Aprovados (via filter) ---
Ana - Media: 8.17
Daniela - Media: 9.50
Amanda - Media: 9.67
Guilherme - Media: 9.67
Matheus - Media: 9.67

```

6. Parte 4 -- Implementacao Logica (Prolog)

Codigo-fonte: parte4_logico.pl

```

1 | % Parte 4 - Implementacao Logica (Prolog)
2 | % Sistema de Analise de Desempenho Academico
3 |
4 | % Fatos: aluno(Nome, Matricula, Nota1, Nota2, Nota3, Frequencia)
5 |
6 | aluno(ana,      2024001, 8.0, 7.5, 9.0, 85).
7 | aluno(bruno,    2024002, 5.0, 6.0, 4.5, 80).
8 | aluno(carlos,   2024003, 7.0, 6.5, 8.0, 60).
9 | aluno(daniela,  2024004, 9.0, 9.5, 10.0, 95).
10 | aluno(amanda,   2024005, 10.0, 9.0, 10.0, 100).
11 | aluno(guilherme, 2024006, 9.0, 10.0, 10.0, 100).
12 | aluno(matheus,  2024007, 10.0, 10.0, 9.0, 100).
13 |
14 | % Regra: calcular media
15 | media(Nome, Media) :-
16 |     aluno(Nome, _, N1, N2, N3, _),
17 |     Media is (N1 + N2 + N3) / 3.
18 |
19 | % Regra: verificar frequencia
20 | frequencia_ok(Nome) :-
21 |     aluno(Nome, _, _, _, _, Freq),
22 |     Freq >= 75.
23 |
24 | % Regra: aprovado
25 | aprovado(Nome) :-
26 |     media(Nome, Media),
27 |     Media >= 6.0,
28 |     frequencia_ok(Nome).
29 |
30 | % Regra: reprovado
31 | reprovado(Nome) :-
32 |     aluno(Nome, _, _, _, _, _),
33 |     \+ aprovado(Nome).
34 |
35 | % Consultas:
36 | %   ?- aprovado(X).
37 | %   ?- reprovado(X).
38 | %   ?- media(ana, M).
```

Exemplo de Consultas e Saida

```

?- aprovado(X).
X = ana ;
X = daniela ;
X = amanda ;
X = guilherme ;
X = matheus.

?- reprovado(X).
X = bruno ;
X = carlos.

?- media(ana, M).
M = 8.166666666666666.
```

7. Comparacao de Legibilidade

Clareza dos nomes:

Todas as versoes Python usam nomes descritivos em portugues. Prolog usa atomos curtos (ana, aprovado), concisos mas claros.

Organizacao:

A OO e a mais organizada: dados e logica encapsulados em classes. A funcional separa entrada/processamento/saida. A imperativa e linear e direta.

Detalhes sintaticos:

Prolog tem a menor quantidade de sintaxe. Python imperativo exige mais linhas por usar while e indices explicitos.

Fluxo do programa:

A imperativa e a mais facil de seguir passo a passo. A funcional exige entender composicao de funcoes. Prolog nao tem fluxo explicito.

Localizacao das regras:

Em Prolog, cada regra e uma clausula isolada. Na OO, estao nos metodos. Na funcional, em funcoes nomeadas.

Conclusao: *Para iniciantes, a imperativa e mais legivel. Para projetos maiores, a OO oferece melhor organizacao. Prolog e o mais conciso, mas exige familiaridade.*

8. Comparacao de Facilidade de Escrita

Quantidade de codigo:

Prolog: ~40 linhas. Funcional: ~70 linhas. Imperativa: ~80 linhas. OO: ~100 linhas.

Dificuldade sintatica:

Python e uniforme nos tres paradigmas. Prolog exige aprender unificacao e backtracking.

Manipulacao de listas:

A funcional e a mais natural com map/filter. A imperativa usa indices manuais. A OO itera com for.

Estruturas auxiliares:

A OO exige criar classes. A funcional usa dicionarios. A imperativa usa listas de listas. Prolog usa fatos diretamente.

Conclusao: *A funcional Python foi a mais rapida de implementar. Prolog e conciso, mas exige mudanca de mentalidade.*

9. Comparacao de Confiabilidade

Verificacao de tipos:

Python tem tipagem dinamica -- erros de tipo so aparecem em tempo de execucao. Prolog nao tem tipos explicitos.

Erros em tempo de execucao:

A imperativa e mais suscetivel a erros de indice (alunos[i][5]). A OO protege dados com encapsulamento. A funcional

reduz efeitos colaterais com funcoes puras.

Acesso a memoria:

Python e Prolog nao permitem acesso direto a memoria, eliminando riscos de buffer overflow (diferente de C).

Testabilidade:

A funcional e a mais testavel: funcoes puras podem ser testadas isoladamente. A OO permite testes por metodo.

Conclusao: A funcional oferece maior confiabilidade por funcoes puras. A OO vem em segundo pelo encapsulamento.

10. Comparacao sobre Nomes, Escopo e Tempo de Vida

Versao Imperativa

Nomes declarados: caminho_arquivo, lista_alunos (globais); nome, media, i (locais).

Visibilidade: globais visiveis em todo o modulo; locais apenas dentro de suas funcoes.

Escopo: funcoes como calcular_media() tem escopo local; parametros nao vazam.

Tempo de vida: locais criadas na chamada e destruidas ao retornar; globais existem durante toda a execucao.

Versao Orientada a Objetos

Nomes declarados: classes Aluno e Turma; atributos self.__nome, self.__nota1; metodos calcular_media().

Visibilidade: atributos com __ (name mangling) visiveis apenas dentro da classe; metodos publicos acessiveis externamente.

Escopo: cada objeto tem seu proprio escopo de atributos; metodos acessam self.

Tempo de vida: objetos existem enquanto houver referencia; garbage collector os destroi quando sem referencias.

Versao Funcional

Nomes declarados: funcoes puras calcular_media(), verificar_aprovacao(), gerar_resultado(); variavel alunos no modulo.

Visibilidade: cada funcao ve apenas seus parametros e variaveis locais. Sem estado mutavel compartilhado.

Escopo: funcoes sao de escopo global; lambdas em filter() capturam variaveis do escopo envolvente (closure).

Tempo de vida: dados criados a cada chamada e descartados apos retorno; listas intermediarias vivem no escopo do modulo.

Versao Logica (Prolog)

Nomes declarados: predicados aluno/6, media/2, aprovado/1; variaveis logicas Nome, Media.

Visibilidade: fatos e regras sao globais; variaveis logicas existem apenas dentro de sua clausula.

Escopo: cada clausula e independente; a variavel Nome em aprovado(Nome) so existe naquela regra.

Tempo de vida: variaveis instanciadas durante unificacao e deixam de existir ao termino da consulta ou backtracking.

11. Comparacao sobre Tipos de Dados

Python (3 versoes)

Tipagem: dinamica e forte.

Tipos primitivos: int, float, str, bool.

Strings: tipo nativo str, imutavel.

Colecoes: listas, dicionarios, tuplas.

Objetos: sim (versao OO com classes Aluno e Turma).

Conversao de tipo: explicita com float(), int().

Erros de tipo: TypeError em tempo de execucao.

Prolog

Tipagem: sem tipos explicitos.

Tipos primitivos: atomos, numeros, variaveis logicas.

Strings: representadas como atomos (ana, bruno).

Colecoes: listas Prolog e termos compostos.

Objetos: nao existe o conceito.

Conversao de tipo: automatica via operador is.

Erros de tipo: falha na unificacao (nao gera excecao, apenas falha).

12. Comparacao entre os Paradigmas

Como a versao imperativa organiza a solucao?

Sequencialmente: le dados, percorre com laço while, calcula media, verifica condicao, imprime. O programador controla cada passo.

Como a versao orientada a objetos organiza a solucao?

Modela o dominio: Aluno encapsula dados e comportamento, Turma gerencia a colecao. A logica e distribuida em metodos.

Como a versao funcional organiza a solucao?

Como pipeline de transformacoes: ler_dados -> map(gerar_resultado) -> imprimir_relatorio. Dados fluem entre funcoes puras.

Como a versao declarativa organiza a solucao?

Declara fatos (dados dos alunos) e regras (media, aprovacao). O motor de inferencia de Prolog encontra as respostas.

Qual versao ficou mais proxima da forma humana?

Prolog. A regra aprovado(Nome) :- media(Nome, M), M >= 6.0 e quase linguagem natural.

Qual versao ficou mais proxima da maquina?

A imperativa. O laço while com indice manual e atribuicoes explicitas reflete como o processador executa instrucoes.

Qual versao parece mais adequada para esse problema?

A funcional. O problema e essencialmente uma transformacao de dados (entrada -> processamento -> saida), caso ideal do paradigma funcional.

13. Conclusao

Cada paradigma demonstrou forcas e limitacoes claras ao resolver o mesmo problema:

O paradigma imperativo e o mais intuitivo para iniciantes e oferece controle total sobre o fluxo, mas gera codigo mais verboso e propenso a erros de indice.

O paradigma orientado a objetos brilha em organizacao e manutenibilidade. Encapsulamento protege os dados e facilita extensoes futuras (ex.: adicionar novos tipos de aluno). E o mais adequado para sistemas grandes.

O paradigma funcional produziu o codigo mais conciso e testavel. Funcoes puras sem efeito colateral facilitam depuracao e paralelismo. E ideal para problemas de transformacao de dados.

O paradigma logico e o mais declarativo e conciso. O programador descreve o que quer, nao como fazer. E poderoso para regras de negocio e sistemas especialistas, mas limitado para I/O e interfaces.

Nao existe paradigma universalmente "melhor". A escolha depende do problema: sistemas corporativos favorecem OO; pipelines de dados favorecem funcional; regras de inferencia favorecem logico; scripts simples favorecem imperativo. O programador competente domina multiplos paradigmas e escolhe o mais adequado para cada situacao.